

Topic 20. Excel VBA: Macros and Basis of VBA (Coursepack p.16-17)

Topic 21. Excel VBA: Loops and conditional statements (Coursepack p. 14-15)

VBA and Macro Basics

What is Visual Basic for Applications (VBA)?

A menu- and icon-based spreadsheet program to solve engineering problems.

The biggest advantage of VBA is the ability to communicate with excel.

Setup:

Windows user - File > Options > Customize Ribbon > Check **Developer**

Next, Developer>Visual Basic>Tools>Options> Check **Require Variable Declaration**

Mac user - Excel > Preferences > Ribbon & Toolbar > Check **Developer**

Next, Developer>Visual Basic>Excel> Preferences > **Check Require Variable Declaration**

Create a new VBA project

Developer->Visual Basic (or simply press Alt F11, or fn + option + F11)

Then, in VBA interface: Insert > Module (note: always program in Module not Microsoft Excel Objects)

What is Macro?

A recorded sequence of operations that can be performed repeatedly.

Record/Run Macro

Developer > Record Macro

Select "Use Relative References" function while "Record Macro" - Run macro near your last cursor position

Insert Button

Developer > Button

To see list of macros:

Developer->Macros (or simply press Alt F8, or fn + option + F8)

Save as .xlsm (Macro-enabled workbook)

VBA Programming (15 example codes)

Option Explicit

' The statement "Option Explicit" enforces that VBA always checks whether variables you use in your code have been declared previously
' using "Dim" as you will see later.
' Go to Tools/Options/ and check "Require variable declaration" for "Option Explicit" to appear each time you open and new Module.

Sub **first_code**()

' Each program/macro starts with the word "Sub" followed by a name. The name is a continuous set of characters except some special characters,
' such as "*", "-", "+", "/", ".", etc. It can contain digits but a digit cannot be placed at the beginning of the name
' (it can be in the middle or at the end of the name)
' Sub refers to "subroutine". Just a jargon.

' The purpose of this code is very simple: sum up two numbers and return the result to the user through a window that pops up on the screen after the calculation is finished.

' The program can be operated from any worksheet in the workbook.

Dim **number1** As Double

Dim **number2** As Double

Dim **result** As Double

' Here we define our "number", i.e., declare variables that we will use throughout the program.
' For our purpose we define variables for each subroutine separately. i.e., the names of the variables defined in this program called
' "first_code" cannot be used in other programs in this module (unless you define them again in that other program.

' There are tricks one can do to declare global variables but we will not get into this exercise at this stage.

' Here we define 3 variables, called number1, number2, and result. All belong to the family of real numbers with double precision.

' The general format is:

' "Dim my_variable As Variable_type", where "my_variable" is the name of variable you pick (cannot be a word reserved by VBA,

' such as "sub", "end", "If", "Then", ect. The "Variable_type" will assign what family of numbers or strings or other types (more below)

' the variable belongs to. We will use primarily the following types of variables:

' **Double** (double precision real number): 8 bytes

```
' Single (single precision real number): 4 bytes
' Integer (integer number): 2 bytes (-32,768 to 32,767)
' Long (large integer number): 4 bytes
' Boolean (logical variable either true or false)
' String (any set of characters btw "&")
'
' more on types: http://www.informit.com/articles/article.aspx?p=339929&seqNum=2
'
' Note that you cannot use two different types for a variable having the same name.
' The variable the way you type it is case insensitive, i.e., "MyMom" is the same as "Mymom" is
the same as "mymom".
```

```
number1 = 4.3
```

```
number2 = 2.5
```

```
' Here we assign values to variables word1 and word2.
' Please note that the sign "=" does not mean equal. It means assign (something like "<="). That
is, everything on the right of the "=" must
' be known and be assigned as a value to the variable on the left of the "=" sign.
```

```
result = number1 * number2
```

```
' Here we perform a simple operation with the two variables, i.e., multiplication.
' What is important here is that the result of the operation is stored in another variable called
result.
```

```
MsgBox (result)
```

```
' MsgBox displays the value of the variable assigned in the argument. Here we display the
content of the variable result.
' The results are communicated to the user through a new window that pops up on the screen.
```

```
End Sub
```

```
' This is the end of the "subroutine". This section is created automatically as soon as you finish
writing the header of the subroutine.
```

How to run the VBA code:

Either in the VBA interface, click the triangle (Run Sub, or F5)
Or in the excel interface, Developer>Macros>Run

```
Sub second_code()
```

' This code asks the user to supply two numbers via keyboard and performs their division.
' The result is reported to the user through a new window that pops up on the screen as soon as the operation is finished.

' The program can be operated from any worksheet in the workbook.

Dim a As Double

Dim b As Double

Dim c As Double

' Here we declare the variables we will use in our program: a, b, and c. All are real numbers with double precision.

a = InputBox("enter value of a")

b = InputBox("enter value of b")

' Instead of typing in the values that would be assigned to the various variables, we can insert them through a menu that opens up on the screen.

' This makes the process on inputting numbers more general (i.e., we do not need to edit the code every time we change the input.

c = a / b

' Here we perform the division a/b and pass the result into a variable c.

MsgBox (c)

' We've seen this before...

End Sub

' All works well as long as a and b are input as numbers and b is a non-zero number. Just like we cannot divide by zero, neither does VBA.

' Thus, if you pick b=0, the program will 'complain' (i.e., it will stop and display an error message.

' Below we write a code that checks whether b=0 and if it is it does not perform the division.

' But before that we will learn how to communicate between the worksheet and the VBA code.

Sub third_code()

' This code asks the user to supply two numbers in cells A1 and B2 in a worksheet and it performs their division.

' The result is reported to the user in cell C1

' Note you need to have Sheet2 active when running this program

Dim a As Double

Dim b As Double

Dim c As Double

' Here we declare the variables we will use in our program: a, b, and c are real numbers.

a = Range("A1")

b = Range("B1")

' Instead of asking the user to supply the number via the code itself (cumbersome), or the keyboard (too much typing), we can read them from the worksheet. And we can then use worksheet to report the results.

' This communication with/within the worksheet (or among various worksheets) makes VBA a very useful tool that simplifies your life and brings Excel to another level of usage.

' We will show two different ways of communicating with the worksheet - one method uses a command called "Range" and it uses as an argument a cell (or a range of cells). The nice thing about this command is that it tells you directly what cell(s) within the worksheet you're operating with.

' The limitation is that the name(s) or the cell(s) is(are) 'hardwired', i.e., they cannot be changed unless one edits the code.

' Well, this is technically not correct and I can show you how to change the name of the cell 'on the fly but we will likely leave it for later.

' Here we demonstrate the use of the command Range to read the contents of cells A1 and B1 and pass it to variables a and b, respectively.

c = a / b

' Here we perform the division a/b and pass the result into a variable c.

Range("C1") = c

' As you may have guessed, this passes the value of the variable c into cell C1. Quite simple, huh?

End Sub

IF/Else statement

Sub fourth_code()

' This code reads two numbers from cells A1 and B2 in a worksheet and it performs their division if b is a non-zero value.

' If b=0, the program returns "N/A". The result is reported to the user in cell C1.

' Note you need to have Sheet2 active when running this program.

Dim a As Double

Dim b As Double

Dim c As Double

' Here we declare the variables we will use in our program: a, b, and c are real numbers.

```
a = Range("A1")
```

```
b = Range("B1")
```

' As before, we read the contents of cells A1 and B1 and pass it to variables a and b, respectively.

' We have seen in the previous code that a/b returns a number if a and b are both numbers and b is non-zero.

' Using b=0 and trying to perform a/b operation will result in a run-time error.

,

' One way to fix this issue is to check whether b is zero or non-zero. If b=0 we will not perform the division but instead report "N/A" into

' the sheet. We only perform the operation if b is not equal to zero.

,

' The statement "If [condition] Then; [action]; End If" will enable to accomplish the task. We test [condition] and then based on the result

' either perform {action} (i.e., if the condition is satisfied) or skip the [action].

' Consider this example:

,

```
' If b <> 0 Then
```

```
'   c = a / b
```

```
'   Range("C1") = c
```

```
' End if
```

,

' In the section above we test whether b is non-zero (b<>0). If so, we perform the operation (c=a/b) and report the result to cell "C1";

' i.e., Range("C1") = c. Do not forget to add "End If" to end of each If statement.

,

' Ok, we have tested whether b is non-zero and if it is we perform the division. We now have to test whether b=0. If so, we cannot divide a by b.

' We accomplish this by the following set of statements:

,

```
' If b = 0 Then
```

```
'   Range("C1") = "N/A"
```

```
' End If
```

,

' Here we simply populate cell "C1" with a statement N/A if b=0. Note that "N/A" has to be inside double quotes so that we transfer the three

' characters (N, /, A) as a continuous set of strings (i.e., letters).

,

' This is all good and we are ready to use it, but it is a little cumbersome. Let's see if we can clean-up the commands a little and write it in

' a more organized fashion.

```
If b <> 0 Then
    c = a / b
    Range("C1") = c
```

```
Else
```

```
    Range("C1") = "N/A"
```

```
End If
```

' This is it! The 6 rows above do the same trick that we described previously but they use a little more complex set of commands:

' "If [condition] Then; [action1]; Else; [action2]; End If"

' The structure should be self-explanatory. Do we need to describe how it works?

```
End Sub
```

```
Sub fifth_code()
```

' The same function as in "fourth_code" except the communication between the VBA code and the worksheet is done via a command "Cells(x,y)",

' where x and y are indices denoting a row and a column in the active worksheet.

' Note that y=1 corresponds to column "A", y=2 corresponds to column "B", y=3 corresponds to column "C", etc.

' Note you need to have Sheet2 active when running this program.

```
Dim a As Double
```

```
Dim b As Double
```

```
Dim c As Double
```

' Here we declare the variables we will use in our program: a, b, and c are real numbers.

```
a = Cells(1, 1)
```

```
b = Cells(1, 2)
```

' If you got all the way down here, this should be easy to understand. With "cells(1,1)" we read the content of the cell located in the

' 1st row and 1st column, that is "A1", and pass its content into variable a. In a similar fashion,

"b = Cells(1, 2)" reads the content of the

' cell "B1" and it gets passed into a variable b.

```
If b <> 0 Then
```

```
    c = a / b
```

```
    Cells(1, 3) = c
```

```
Else
```

```
    Cells(1, 3) = "N/A"
```

```
End If
```

' This is the same as before, except we use the command "Cells(x,y)" to communicate the result to the worksheet.

' Thus "Cells(1, 3) = c" writes the content of the variable c into a cell located in the 1st row, 3rd column, i.e., "C1".

End Sub

For/Next loop

Sub sixth_code()

' The same function as in "fifth_code" except we perform the very same operation on a series of numbers (i.e., not just a single couple).

' The couples of organized such that each couple of numbers that will be divided occupies columns "A" and "B" and each couple is present in a

' separate row. In this first example we know how many couples are present in the worksheet. Later we extend our approach to situations that

' involve a previously-unknown number of couples of a and b.

' Note you need to have Sheet2 active when running this program.

Dim a As Double

Dim b As Double

Dim c As Double

Dim i As Integer

' Here we declare the variables we will use in our program. a, b, and c are real numbers, i is an integer.

For i = 1 To 5

' If you notice, we have a new set of commands here: "For i=x To y; [action]; Next i". Here i is an integer number that acts as a counter

' running from value x (in our example 1) to value y (in our example 4).

' [action] is the same set of commands we used in our previous code, except instead of 'hardwiring' the row index in the command "Cells",

' we use i, our counter variable, as the row index. Each time we start a new cycle, the value of i increments by 1.

a = Cells(i, 1)

b = Cells(i, 2)

If b <> 0 Then

c = a / b

Cells(i, 3) = c

Else


```
Cells(i, 3) = "N/A"  
End If
```

```
Next i
```

```
' This is where the cycle ends.  
,
```

```
' If we want, we can change the increment in i. For instance, "For i = 1 To 19 Step 2" will let i  
assume values of 1, 3, 5, 7,..., 17, 19.
```

```
' Similarly, writing "For i = 5 To 2 Step -1" will let i change from 5, to 4, to 3, and to 2 each time  
we run the cycle.
```

```
End Sub
```

Jump out the loop by the GoTo command

```
Sub seventh_code()
```

```
' The same function as in "sixth_code". We have to perform the same operation (i.e., division  
of two numbers) for previously unknown sets of two
```

```
' numbers present in the worksheet. This is not the best way to solve the problem, however.  
We still use it by assuming that the number of
```

```
' couples we have to perform the division for is not larger than 1000. It may or may not be true.  
In principle we can make it 'more safe' by
```

```
' setting the upper bound in the "For" cycle to a value larger than 1000. But if the number of  
couples present in the worksheet exceeds this
```

```
' boundary we will not evaluate the division for all couples. The second complication is with the  
GoTo command (see below). This is not a very
```

```
' clean way to solve the problem (i.e., programmers often avoid using too many GoTo  
commands as it makes the program more challenging to comprehend.
```

```
' Nevertheless, we start with this code as a simple modification to the previous code  
(sixth_code) to make it more general.
```

```
' Note you need to have Sheet2 active when running this program.
```

```
Dim a As Double
```

```
Dim b As Double
```

```
Dim c As Double
```

```
Dim i As Integer
```

```
' Here we declare the variables we will use in our program. a, b, and c are real numbers, i is an  
integer.
```

```
For i = 1 To 1000
```

```
' We set the upper boundary in the "For" cycle to a value that we think is larger than the  
number of couples of numbers for which we want to
```

' perform the division operation. We will exit the cycle as soon as we encounter an empty cell in column A.

If Cells(i, 1) = "" Then

GoTo label1

End If

' This is where we check whether the cell in the A column is empty or has a value (we assume that the cell, if not empty is populated with a

' number. The command "If Cells(i, 1) = "" checks if the cell in column A is empty.

' If it is, the program jumps out of the "For...Next" loop into a location indicated by the name specified just after the "GoTo" statement.

' This can be any name (does not have to be define/declared) as long as it is not a name that would coincide with a pre-defined variable or a

' reserved "word" used by VBA. We use the name label1, but can be anything else... We just have to remember adding the name outside the loop

' (see below).

a = Cells(i, 1)

b = Cells(i, 2)

If b <> 0 Then

c = a / b

Cells(i, 3) = c

Else

Cells(i, 3) = "N/A"

End If

Next i

' This section is identical to what we wrote before...

label1:

' This is the point in the program to which we 'jump' into when exiting the loop. The name has to be the same as in the "GoTo" statement used

' in the loop before.

End Sub

Do/Loop & jump out by Exit Do

Sub eight_code()

' This is the correct way to perform an 'infinite loop'. It uses an infinite repetition using "Do... [action]... Loop" statement.

' When using the Do/Loop cycle we need to make sure that we can exit the cycle at some point.

' If we don't, the cycle will continue to go on indefinitely. It is very easy to make a mistake while using this loop (I have done it multiple times). I recommend that you save your work before running a code with this loop. If there is no safe way to exit the loop, the program will continue on running and the only way to stop it is to kill the program. If you did not save your work before running the loop, you will lose it.

' Note you need to have Sheet2 active when running this program.

Dim a As Double

Dim b As Double

Dim c As Double

Dim i As Integer

' Here we declare the variables we will use in our program. a, b, and c are real numbers, i is an integer.

i = 1

' We use i as a counter that will be increment by +1 each time we run a new cycle. We initially start with i=1...

Do

If Cells(i, 1) = "" Then

Exit Do

End If

' The same testing as before. If the cell in column A does not contain a number (and is empty) we simply exit the loop.

' This is the safety statement in our code. Without it the program will continue on running (actually, it will crash when encountering no value...)

a = Cells(i, 1)

b = Cells(i, 2)

If b <> 0 Then

c = a / b

Cells(i, 3) = c

Else

Cells(i, 3) = "N/A"

End If

' This section is identical to what we wrote before...

i = i + 1

' This line is very important. It enables to increment the values of the variable "i" by 1 so that it can be used in the next cycle.

' If we did not increment i by +1 the program will continue exploring cells A1 and B1 and performing the division operation indefinitely.
' No way to stop it... other than killing the code...

Loop

' This is the end of the "Do...Loop" cycle.

End Sub

Sub ninth_code()

' In this code we sum up numbers from 1 to 100 and report the result of the operation using a message box.

' The program can be operated from any worksheet in the workbook.

Dim sum100 As Integer

Dim i As Integer

' Here we declare the variables we will use in our program. sum100 and i are both integers. The variable sum100 contains partial summation

' results (as well as the final value of the entire sum of the number from 1 to 100; the variable i is use as a counter that ranges from 1 to 100.

sum100 = 0

' It is always good to zero your variables (although they should all be zeroed at the time of their creation...

For i = 1 To 100

 sum100 = sum100 + i

Next i

' Here we add all numbers starting from 1 all the way to 100 into the variable sum100 that gets incremented by a new value of i each time the

' cycle gets performed...

MsgBox (sum100)

' Simply displaying the final result of the summation.

End Sub

Use pre-defined Excel Functions in VBA

Sub tenth_code()

' We mentioned in class that VBA and Excel use their own separate sets of pre-defined functions (sadly).

' Yet, we tap into the very many pre-defined functions own to Excel even from within VBA by using "Application.WorksteetFunction.PICK_THE_FUNCTION"

' command. Here we demonstrate the usage of this approach in evaluating both the natural and decadic logarithms of a given argument. In our example,

' the argument is stored in cell A1 and we read its content into a variable b. We then perform the operation of the argument using both Excel-

' as well as VBA-based functions. We then store the result in different cells in the worksheet.

' See below for more details.

' Note you need to have Sheet3 active when running this program.

Dim a As Double

Dim b As Double

' Here we declare the variables we will use in our program. The variable b is the argument of the logarithm we plan to evaluate, the variable a

' contains the result of the operation.

a = Cells(1, 1)

b = Application.WorksheetFunction.Ln(a)

Cells(1, 2) = b

' We read the argument from the cell A1 and store is in a variable b. We then perform the calculation of the natural logarithm of b using the

' Excel "Ln" function and store it in a cell B1 (using Cells(1,2)).

b = Application.WorksheetFunction.Log10(a)

Cells(1, 3) = b

' We also calculate the decadic logarithm of b using Excel's "Log10" function and store the result in a cell C1 (using Cells(1,3)).

b = Log(a)

Cells(1, 4) = b

' Here we calculate the natural logarithm of b using a VBA-own function called "Log". We return the result in a cell D1 (using Cells(1,4)).

b = Log(a) / Log(10)

Cells(1, 5) = b

' VBA does not know how to calculate the decadic logarithm of a number. We can still calculate it by recognizing that $\log_{10}(y) = \log(y) / \log(10)$.

' We do so here and store the result in a cell E1 (using Cells(1,5)).

End Sub

Sub eleventh_code()

' In this example we test whether a number read from the spreadsheet is negative, positive or zero. We assume that the total numbers of the

' entries (i.e., numbers) is known a priori.

' Note you need to have Sheet4 active when running this program.

Dim i As Integer

Dim mynumber As Double

' Here we declare the variables we will use in our program.

For i = 1 To 7

 mynumber = Cells(i, 1)

 If mynumber < 0 Then

 Cells(i, 2) = "negative"

 End If

 If mynumber > 0 Then

 Cells(i, 2) = "positive"

 End If

 If mynumber = 0 Then

 Cells(i, 2) = "zero"

 End If

Next i

' We test each of the numbers by using multiple "if" statements. The nomenclature should be obvious. We report the results by passing a

' "string" of words into the appropriate cell.

End Sub

Alternative code:

Sub eleventh_code_v2()

Dim i As Integer

Dim mynumber As Double

For i = 1 To 7

 mynumber = Cells(i, 1)

 If mynumber < 0 Then

 Cells(i, 2) = "negative"

```
Elseif mynumber > 0 Then
    Cells(i, 2) = "positive"
Elseif mynumber = 0 Then
    Cells(i, 2) = "zero"
End If
Next i
```

End Sub

Select Case Statement

```
Sub twelveth_code()
```

```
' In this example we test whether a number read from the spreadsheet is negative, positive or
zero. We assume that the total numbers of
' the entries (i.e., numbers) is known a priori.
```

```
' Note you need to have Sheet4 active when running this program.
```

```
Dim i As Integer
```

```
Dim mynumber As Double
```

```
' Here we declare the variables we will use in our program.
```

```
For i = 1 To 7
```

```
    mynumber = Cells(i, 1)
```

```
    Select Case mynumber
```

```
        Case Is < 0 ' Use the "Is" keyword to compare values
```

```
            Cells(i, 2) = "negative"
```

```
        Case Is > 0
```

```
            Cells(i, 2) = "positive"
```

```
        Case Is = 0
```

```
            Cells(i, 2) = "zero"
```

```
    End Select
```

```
Next i
```

```
' Same as before except that we replace the multiple "if" statements with a "Select Case"
command.
```

```
' Compare this to what we used in the previous code... You will immediately see the advantage
of using "Select Case" for multiple entries...
```

```
End Sub
```

```
Sub thirteenth_code()
```

```
' The very same example as in code "twelveth_code" except that we assign the result of the  
operation to a 'string' variable, which is,  
' in turn, reported in the code.
```

```
' Note you need to have Sheet4 active when running this program.
```

```
Dim i As Integer
```

```
Dim mynumber As Double
```

```
Dim mystring As String
```

```
' Here we declare the variables we will use in our program.
```

```
For i = 1 To 7
```

```
    mynumber = Cells(i, 1)
```

```
    Select Case mynumber
```

```
        Case Is < 0
```

```
            mystring = "negative"
```

```
        Case Is > 0
```

```
            mystring = "positive"
```

```
        Case Is = 0
```

```
            mystring = "zero"
```

```
    End Select
```

```
    Cells(i, 2) = mystring
```

```
Next i
```

```
End Sub
```

Nested Loops for filling a 5x5 matrix

```
Sub fourteenth_code()
```

```
' Demonstration of nested loops. The loops cannot cross each other. The example also shows  
how one can use a number (variable i) to convert it
```

```
' to a 'string' of letters using the command 'Str'. We use the command 'Trim' to remove the  
empty space in front of a positive number.
```

```
' Note you need to have Sheet5 active when running this program.
```

```
Dim i As Integer
```

```
Dim j As Integer
```

```
' Here we declare the variables we will use in our program.
```



```

For i = 1 To 5
  For j = 1 To 5
    Select Case j
      Case 1
        Cells(i, j) = "A" + Trim(Str(i))
      Case 2
        Cells(i, j) = "B" + Trim(Str(i))
      Case 3
        Cells(i, j) = "C" + Trim(Str(i))
      Case 4
        Cells(i, j) = "D" + Trim(Str(i))
      Case 5 ' Alternatively, you can use "Case Else" here
        Cells(i, j) = "E" + Trim(Str(i))
    End Select
  Next j
Next i
' Note that the cycle for 'j' is embedded within the cycle for 'i'.

End Sub

```

Newton-Raphson in VBA

```

Sub fifteenth_code()
' This code calculates the solution to an equation  $f(x)=x+\cos(x)$  using the Newton-Raphson
method. We first determine the solutions using the
' calculation in spreadsheet ("sheet7"). Now we try to recover the same solution using VBA.

' Note you need to have Sheet7 active when running this program.

Dim fxn As Double
Dim f_deriv As Double
Dim x_old As Double
Dim x_new As Double
Dim delta As Double
' Here we declare the variables we will use in our program.

x_old = Cells(4, 8)
' Read the initial guess for x from cell H3.

Do

```

' start infinite loop from which you will exit by using the condition given next to the "Loop" statement

fxn = x_old + Cos(x_old)

deriv = 1 - Sin(x_old)

x_new = x_old - fxn / f_deriv

' Calculate the next value of "x" (i.e., x_new) using the Newton-Raphson formula.

delta = Abs(x_new - x_old)

' Compare the previous value of "x" to the new value of "x". If the difference is small enough, you're done.

x_old = x_new

' Assign the old value of "x" to become a new value of the same.

' One possible way to exit the loop is to use the "If" statement.

' If delta < 0.0000001 Then

' Exit Do

' End If

' Loop

' This is fine... Here we demonstrate how one can "jump out of the loop" by adding the exist condition in the "Until" statement.

Loop Until delta < 1E-07

' Exit the loop when Abs(x_new-x_old) is less than 1e-7

Cells(4, 9) = x_new

' Report the final answer in cell I3.

End Sub